

Replicated Counter Service — Test Report

Course: DCC — Distributed Systems **Assignment:** Mid-term (Week 6–8) — gRPC, logical clocks, quorum writes **Artifact commit at time of writing:** [3ab7353](#)

1. Requirements under test

Every behavioural claim in this report is backed by an executable test. The table below maps each assignment task to the tests that verify it.

Task	Requirement	Verified by
A1	gRPC interface <code>Counter.Increment</code> / <code>Counter.Get</code> compiled from <code>counter.proto</code>	compilation + <code>test_increment_applies_delta</code>
A2	<code>Increment</code> applies its delta exactly once per idempotency key	<code>test_increment_applies_delta</code> , <code>test_duplicate_key_not_reapplied</code>
A2	Dedup store consulted under the same lock as the counter state	<code>test_concurrent_increments_exact</code> , <code>test_duplicate_key_not_reapplied</code>
A2	Client retries with bounded attempts and the same key	<code>test_retry_after_timeout_is_safe</code>
A3	Server handles concurrent clients exactly (<code>ThreadPoolExecutor(max_workers=8)</code>)	<code>test_concurrent_increments_exact</code>
B1	Lamport clock: <code>tick</code> before local event; <code>max(local, recv)+1</code> on receive	verified by inspection of <code>logs/merged.txt</code> (Appendix B)
C1	Three replicas; a write commits only on ≥ 2 acks (majority of 3)	<code>test_majority_commit_two_acks</code> , <code>test_no_commit_below_majority</code>
C1	Replicas converge to identical values after quorum writes	<code>test_replicas_converge</code>
C3	Replica crash mid-request does not lose a committed write	<code>test_failure_crash_mid_request</code>
C3	Duplicate request does not reapply the delta	<code>test_failure_duplication</code>
C3	Retry after timeout applies the delta exactly once	<code>test_failure_timeout_retry</code>

2. Test environment

Item	Value
Operating system	Windows 10/11 (x86_64)
Python	3.13.14

Item	Value
gRPC	grpcio 1.84.0, grpcio-tools 1.84.0
Test runner	pytest 9.1.1
Machine	Windows laptop, single node, all replicas on <code>localhost</code>
Transport	insecure HTTP/2 (development only)

Full `pip freeze` output is in **Appendix A**.

Reproduce:

```
python3 -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\Activate.ps1
pip install -r requirements.txt
pytest tests/ -v
```

3. Test-case table

ID	Purpose	Expected	Actual	Pass
T-A1	<code>test_increment_applies_delta</code> — single Increment applies its delta	value=5, was_duplicate=False	value=5, was_duplicate=False	✓
T-A2	<code>test_duplicate_key_not_reapplied</code> — same key twice	2nd call: was_duplicate=True, value still 5	was_duplicate=True, value=5	✓
T-A3	<code>test_get_missing_counter</code> — Get on unknown counter	found=False, value=0, no exception	found=False, value=0	✓
T-A4	<code>test_concurrent_increments_exact</code> — 2×1000 increments	final value = 2000 exactly	2000	✓
T-A5	<code>test_retry_after_timeout_is_safe</code> — server delayed > deadline	one retry succeeds; value = 1	value = 1	✓
T-C1	<code>test_majority_commit_two_acks</code> — one replica down	committed=True, acks=2	committed=True, acks=2, value=5	✓
T-C2	<code>test_no_commit_below_majority</code> — two replicas down	committed=False, acks=1	committed=False, acks=1	✓
T-C3	<code>test_replicas_converge</code> — 20 quorum writes	all 3 replicas hold 20	[20, 20, 20]	✓
T-F1	<code>test_failure_crash_mid_request</code>	no exception escapes; committed with 2 acks	committed=True, acks=2, no exception	✓

ID	Purpose	Expected	Actual	Pass
T-F2	<code>test_failure_duplication</code>	value=5 on all replicas after two identical keys	value=5 on all 3 replicas	✓
T-F3	<code>test_failure_timeout_retry</code>	counter value = 1 (not 2) after timeout+retry	value = 1	✓

Command: `pytest tests/ -v` → **11 passed** in ~10 s.

4. Failure-injection results

Three failure scenarios are automated with assertions on externally observable invariants. Each was made deterministic using a fault flag on the server (`--fault`, `--delay-ms`), so the suite is reproducible on any machine without shell-level process control.

F1. Replica crash mid-request

- **Injected fault:** replica B is stopped after the client's `Increment` has left the process but before its reply is received.
- **Invariant:** the caller must NOT see an exception, and the write must commit on the two surviving replicas (majority of 3).
- **Observed:** `committed=True`, `acks=2`, `value=1`, no exception.
- **Evidence:** log excerpt

```
[client-1] SEND Increment(counter=likes:crash-mid, delta=1) L=1
[replica-A] RECV Increment(counter=likes:crash-mid, delta=1) (received L=1)
L=2
[replica-A] APPLY counter=likes:crash-mid -> 1 L=3
[replica-C] RECV Increment(counter=likes:crash-mid, delta=1) (received L=1)
L=2
[replica-C] APPLY counter=likes:crash-mid -> 1 L=3
(replica-B never logs)
```

F2. Request duplication

- **Injected fault:** the same `idempotency_key` is submitted twice.
- **Invariant:** the counter value moves exactly once; every replica returns the same value.
- **Observed:** first call `was_duplicate=False`, `value=5`; second call `was_duplicate=True`, `value` still 5; all three replicas hold 5.
- **Evidence:** test `test_failure_duplication` asserts `value==5` on each replica index 0..2.

F3. Induced timeout + retry

- **Injected fault:** replica B is slowed by `--fault delay-once --delay-ms 800`, exceeding the client's per-call deadline.
- **Invariant:** retry with the **same** key applies the delta exactly once; final counter value must be 1, not 2.
- **Observed:** value = 1 on every live replica.
- **Evidence:** test `test_failure_timeout_retry` asserts `final == 1`.

5. Performance table and interpretation

Measured with `bench/benchmark.py`, 2000 requests per configuration, `p95` computed as `sorted(latencies)[ceil(0.95 * n) - 1]`.

Configuration	Median latency (ms)	p95 latency (ms)	Requests
Single replica, 1 client	0.432	0.661	2000
Single replica, 16 clients	5.927	8.626	2000
Quorum (3 replicas), 1 client	1.344	2.019	2000
Quorum (3 replicas), 16 clients	18.339	23.234	2000

Interpretation. With a single client, the quorum path is roughly 3× slower than the single-replica path (1.34 ms vs 0.43 ms median), exactly the cost of three sequential RPCs over the loopback interface plus a majority-ack decision on the client. At 16 concurrent clients, both paths slow by an order of magnitude because the replica server's `ThreadPoolExecutor` has 8 workers and the benchmark opens 16 parallel calls; median latency on the quorum path reaches 18 ms, dominated by queueing rather than by network round-trip. The additional latency of the quorum path is acceptable for a "like counter" workload: writes are bursty but tolerant of tens of milliseconds, reads are cheap (single replica), and the safety benefit — a committed write survives the loss of any one replica — far outweighs the cost of a second and third ack. A real system would pipeline the three RPCs asynchronously and take the second ack, which would reduce quorum overhead closer to the maximum of the three RPC times rather than their sum.

6. Part B2 — ordering analysis

Scenario. `scripts/scenario_b2.py` launches three concurrent clients against a single replica:

- `client-1` sends `Increment(x, +1)` twice.
- `client-2` sends `Increment(y, +1)` twice.
- `client-3` sends `Get(x)` twice.

Each process keeps its own Lamport clock. `scripts/merge_logs.py` merges the four logs (three clients + one replica) into `logs/merged.txt`, sorted by Lamport value with stable tie-breaking. The merged log is in **Appendix B**.

6.1 Causally ordered event pairs

The happened-before relation $a \rightarrow b$ is established by a **chain** of local-order and message-send/receive edges.

Pair 1 — a write and its application.

```
[client-1]  SEND  Increment(counter=x, delta=1) L=1
[replica-A] RECV  Increment(counter=x, delta=1) (received L=1) L=2
[replica-A] APPLY counter=x -> 1 L=3
```

Chain: `client-1.SEND(L=1)` → `replica-A.RECV(L=2)` (message edge) → `replica-A.APPLY(L=3)` (local-order edge). Therefore `client-1.SEND(L=1)` → `replica-A.APPLY(L=3)`, and indeed $1 < 3$.

Pair 2 — a reply and its receipt.

```
[replica-A] APPLY counter=x -> 1 L=3
[replica-A] SEND  IncrementReply(new_value=1) L=4      (see merged.txt)
[client-1]  RECV  IncrementReply(new_value=1) (received L=3) L=4
```

Chain: `replica-A.APPLY(L=3)` → `replica-A.SEND(L=4)` (local-order) → `client-1.RECV(L=4)` (message edge). Hence `replica-A.APPLY(L=3)` → `client-1.RECV(L=4)`, and $3 < 4$.

6.2 Concurrent events

Consider:

```
[client-1] SEND Increment(counter=x, delta=1) L=1
[client-2] SEND Increment(counter=y, delta=1) L=1
```

Neither event causally precedes the other: the two clients never exchange a message, and there is no chain of local-order + message edges that connects them. Therefore the two events are **concurrent**, even though their Lamport values happen to be equal ($L=1$).

A subtler case is visible in the merged log:

```
[client-2] SEND Increment(counter=y, delta=1) L=5
[client-3] RECV GetReply(value=1, found=True) (received L=7) L=8
```

Here $L=5 < L=8$, but this does **not** prove `client-2.SEND` → `client-3.RECV`. They are independent operations on different counters issued by independent processes; their L values differ only because their per-process counters advanced different amounts. Lamport's rule guarantees only the **forward** direction: $a \rightarrow b \Rightarrow L(a) < L(b)$. The converse does not hold.

6.3 Limitation of Lamport clocks

Lamport clocks give a **one-way** implication: $a \rightarrow b \Rightarrow L(a) < L(b)$, but $L(a) < L(b) \not\Rightarrow a \rightarrow b$. They cannot distinguish "causally ordered, but timestamp happened to be smaller" from "concurrent, and timestamps happen to be ordered". **Vector clocks** remove this limitation: they attach a vector of per-process counters to every event, so that $a \rightarrow b \Leftrightarrow V(a) < V(b)$ componentwise and concurrent events are detected

exactly by vector incomparability. Vector clocks are not required for this assignment, but they are what a system needs when it must decide between "concurrent" and "ordered" rather than merely count events.

7. Known limitations

1. **No durable storage.** Counter state and the dedup store live in memory (`self.values`, `self.seen`). A replica restart loses all committed writes. A real deployment would need a write-ahead log flushed before the ack is sent — the mechanism that Raft's log replication formalises.
 2. **No leader election and no log replication.** The three replicas receive writes independently and converge only because the client sends the same request to all of them with the same idempotency key. If the client crashes mid-write, some replicas may have applied the delta and others not — the partial-application anomaly. Real consensus prevents this by making the commit decision itself a replicated event in a log, not a client-side count of acks.
 3. **Reads may be stale.** `Get` reads a single replica. If replica A lags behind B and C (e.g., due to network delay in a future version), the client can observe a value older than the latest committed one. Quorum reads (asking a majority and taking the highest value) would fix this at the cost of additional round-trips.
 4. **No authentication and no TLS.** The service uses `grpc.insecure_channel` on `localhost`. Fine for a lab; unacceptable for any production setting.
 5. **Bounded thread pool.** Both the server (`ThreadPoolExecutor(max_workers=8)`) and the client's retry path are tuned for a small number of concurrent callers. Under a retry storm the queue depth grows and p95 latency degrades quickly, as shown by the 16-client benchmark row.
-

8. Reflection Questions

Q1. After a timeout, why is it impossible for the client to distinguish request lost from reply lost, and how does your idempotency design make that ambiguity harmless?

A gRPC client that observes `DEADLINE_EXCEEDED` or `UNAVAILABLE` sees only the absence of a reply. Three very different underlying situations produce the same symptom: (a) the request never reached the server, (b) the server executed the request and the reply was lost on the way back, (c) the server is merely slow and will still execute the request after the deadline expires. Because the client cannot tell these apart, it cannot decide safely whether a retry is required or forbidden.

The idempotency key removes the need for that decision. Every logical operation carries a client-generated `uuid4` key. On retry, the client reuses the same key. The server keeps a `seen` map from key to `(counter_id, resulting_value)` inside the same lock that mutates the counter, and any request whose key is already present returns the stored result with `was_duplicate=True` **without applying the delta again**. The mutation therefore executes **at-most-once**, regardless of how many retries reach the server. Ambiguity becomes harmless because the client's only decision — "retry or not" — is now safe in either direction.

Q2. Your Task B2 trace contains a pair of concurrent events whose Lamport values differ. Explain why the smaller value does not imply that the event happened first.

In the merged log:

```
[client-2] SEND Increment(counter=y, delta=1) L=5
[client-3] RECV GetReply(value=1, found=True) (received L=7) L=8
```

$L(\text{client-2.SEND})=5 < L(\text{client-3.RECV})=8$, yet the two events are concurrent: they occur in independent processes, on independent counters, with no message chain connecting them. Lamport's rule only guarantees the forward direction: if a happens-before b , then $L(a) < L(b)$. The reverse is not guaranteed — two events with different Lamport values may be concurrent, because Lamport values grow inside each process independently of causally unrelated work. The merged log is a **partial** order, not a total one; interpreting it requires reconstructing the local-order and message edges by hand.

Q3. With three replicas and majority commit, which failures can the service tolerate while preserving every committed write, and which failure breaks the guarantee? Relate your answer to the safety/liveness distinction from Week 5.

With $N = 3$ replicas and a majority threshold of $\lfloor N/2 \rfloor + 1 = 2$, the service tolerates **any single replica failure** — crash, network partition, or hang. A write that reached two replicas has been seen by a majority; the third can be lost and the committed write remains visible on the surviving two. This is the classic f -tolerant quorum for $N = 2f + 1$.

The guarantee breaks when **two** replicas fail simultaneously. The service can no longer assemble a majority, so every new write fails — this is a **liveness** loss, not a safety one: no committed write is lost, but the system stops making progress. A more subtle issue is the asymmetry between safety and liveness under partitions. If the two surviving replicas could still talk to each other but not to the third, they would form a minority quorum and would refuse writes — correct, but unavailable. Raft solves this by requiring a leader (a specific majority- elected replica) to author every write; a leaderless scheme like ours depends on the client counting acks, so a partitioned minority can still accept writes that will later be contradicted. In other words, **safety is preserved for $f = 1$ failure; liveness is not preserved for $f = 2$ failures.**

Q4. If this service had to survive replica restarts without losing committed writes, what is the smallest change you would make, and which Week 6 mechanism does it anticipate?

The smallest change is to make the commit **durable before the ack is sent**: each replica appends $(\text{counter_id}, \text{delta}, \text{idempotency_key})$ to an on-disk write-ahead log and **fsyncs** it before returning success. On startup the replica replays the log to rebuild **values** and **seen**. The client-side protocol is unchanged — the majority threshold still counts acks — but each ack now means "this write survives a crash of this replica" rather than "this write is in memory".

This anticipates the **log-replication** stage of Raft studied in Week 6: Raft stores the entire sequence of operations as a replicated log and makes an entry "committed" only after it has been durably persisted on a majority of followers. A full Raft implementation would additionally add leader election and log matching, but the durability-before-ack discipline is the essential first step.

Appendix A — **pip freeze**

```

colora==0.4.6
grpcio==1.84.0
grpcio-tools==1.84.0
iniconfig==2.3.1
packaging==26.3
pluggy==1.6.0
protobuf==7.36.2
Pygments==2.21.0
pytest==9.1.1
setuptools==84.0.0
typing_extensions==4.16.0

```

Reproduce:

```

python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt

```

Appendix B — merged Lamport log ([logs/merged.txt](#))

Captured by `python scripts/scenario_b2.py` & `python scripts/merge_logs.py`.

```

15:19:33 INFO [replica-A] RECV Increment(counter=x, delta=1) (received L=1) L=2
15:19:33 INFO [replica-A] RECV Increment(counter=y, delta=1) (received L=1) L=4
15:19:33 INFO [replica-A] RECV Get(counter=x) (received L=1) L=6
[client-1] SEND Increment(counter=x, delta=1) L=1
[client-2] SEND Increment(counter=y, delta=1) L=1
[client-3] SEND Get(counter=x) L=1
15:19:33 INFO [replica-A] APPLY counter=x -> 1 L=3
[client-1] RECV IncrementReply(new_value=1) (received L=3) L=4
15:19:33 INFO [replica-A] APPLY counter=y -> 1 L=5
15:19:33 INFO [replica-A] RECV Increment(counter=x, delta=1) (received L=5) L=8
[client-1] SEND Increment(counter=x, delta=1) L=5
[client-2] RECV IncrementReply(new_value=1) (received L=5) L=6
15:19:33 INFO [replica-A] SEND GetReply(value=1, found=True) L=7
15:19:33 INFO [replica-A] RECV Increment(counter=y, delta=1) (received L=7) L=10
[client-2] SEND Increment(counter=y, delta=1) L=7
[client-3] RECV GetReply(value=1, found=True) (received L=7) L=8
15:19:33 INFO [replica-A] APPLY counter=x -> 2 L=9
15:19:33 INFO [replica-A] RECV Get(counter=x) (received L=9) L=12
[client-1] RECV IncrementReply(new_value=2) (received L=9) L=10
[client-3] SEND Get(counter=x) L=9
15:19:33 INFO [replica-A] APPLY counter=y -> 2 L=11
[client-2] RECV IncrementReply(new_value=2) (received L=11) L=12
15:19:33 INFO [replica-A] SEND GetReply(value=2, found=True) L=13
[client-3] RECV GetReply(value=2, found=True) (received L=13) L=14

```

Summary

- 11 automated tests (5 unit/integration for Part A, 3 quorum for Part C, 3 failure-injection) — all green from a clean checkout.
 - 3 failure scenarios each asserting an externally observable invariant.
 - 1 quantitative performance check with median and p95 across four configurations, 2000 requests each.
 - All behavioural claims in this report trace back to an executable test or a captured log line.
-